Algorithmique des graphes (2)

Sources :
- Erickson, Algorithms
- Cormen, Algorithmique

Table des matières

I Rappels sur les parcours 2
- I.1 Principes 2
- I.2 Parcours en profondeur décoré 4

II Calcul de composante fortement connexe 10
- II.1 Rappels sur les composantes fortements connexes 10
- II.2 Algorithme de Kosaraju 11

III Arbres couvrant minimaux 17
- III.1 Arbres couvrants 17
- III.2 Algorithme de Kruskal 18
- III.3 Structure union-find 20
  - III.3.a Implémentation naïve 21
  - III.3.b Implémentation arborescente 22
  - III.3.c Implémentation arborescente avec union par hauteur 22
  - III.3.d Implémentation arborescente avec compression de chemin et union par rang 24

IV Calcul du plus court chemin à origine et destination unique 26
- IV.1 Rappels sur l'algorithme de Dijksta 26
- IV.2 Algorithme **A*** 29
- IV.3 Heuristiques admissibles et monotones 30

V Couplages dans les graphes biparis 36
- V.1 Problème de couplage 36
- V.2 Algorithme de Berge 37
- V.3 Le cas des graphes bipartis 40

I Rappels sur les parcours

Rappel :

- Dans un graphe non-orienté (S,A)

$$\sum_{s \in S} \deg(s) = 2|\mathrm{A}|$$

- Dans un graphe orienté (S, A), avec $\deg_{+(-)}$ le degré sortant (entrant)

$$\sum_{s \in S} \deg_+(s) = \sum_{s \in S} \deg_-(s) = |A|$$

I.1 Principes

Algorithme générique de parcours « Whatever-first-search »
Entrée : G = (S,A) un graphe ; $s \in S$
Effet : parcours de G à partir de s

```
Parcours(G,s)
1.  Initialiser une structure de marquage
1.  Marquer les sommets de G comme étant non vus
2.  Initialiser un sac
2.  Mettre s dans le sac
2.  Tant que le sac est non-vide{
2.    Retirer un sommet v du sac
1.    Si v n'est pas vus{
1.      Marquer v comme vus
3.      Pour tout voisin u de v{
2.        Mettre u dans le sac
        }
      }
    }
```

Proposition 1: Les sommets accessibles dans G depuis s sont exactement les sommets marqués comme vus lors du Parcours(G, s)

La complexité de Parcours (G, s) avec G = (V,E) si

- les opérations d'insertion et de suppression dans le sac se font en $O(k)$ avec $k = |\mathrm{sac}|$
- Le graphe est représenté par listes d'adjacence
- le marquage se fait via un tableau de taille |V|

est en $O(|\mathrm{V}| + k|\mathrm{E}|)$

🔥 ***Remarque***

La complexité dépend
- 1. De l'implémentation de la structure de marquage
- 2. De l'implémentation du sac
- 3. De l'imprémlentation du graphe

Démonstation:

- Terminaison : le variant (nb de sommets non vus, |sac|) décroit strictement pour l'ordre lexicographique au cours de la boucle « Tant que ».
  Comme cet ordre est bien fondé sur N^2
- ($\Longrightarrow$) Montrons par récurrence sur $n \in N$, H(n) : « si u est acessible depuis s via un plus court chemin à n arrêtes alors u est marqué comme< vu »
  - Si $n = 0$, il suffit de montrer que s est marqué, et c'est le cas : cf. déroulé de l'algo
  - Soit n tel que H(n). Soit u un sommet à distance $n + 1$ de s.

    Alors considérons v le sommet précédant u dans un plus court chemin de s vers u. Le chemin de s à v est un plus court chemin (sinon, le plus court chemin de s à u n'aurait pas une longueur $n + 1$)

    On applique alors l'hypothèse de récurrence à v : il a été marqué comme vu. D'après l'algo, on a effectué les instructions l.9-11 pour v. En particulier, on a ajouté son voisin u au sac et comme u en est sorti (le sac est vide à la fin), il a été marqué
- ($\Longleftarrow$) Par l'absurde, supposons qu'il existe v un sommet marqué comme vu et non accessible depuis s.
  Prenons v le 1er tel sommet qui entre dans le sac.
  Comme s est accessible depuis s, v $\neq$ s, donc v a été ajouté au sac lors du traitement d'un sommet u dont v est voisin.
  u est nécessairement accessible depuis s par définition de v (s'il ne l'était pas, v ne serait pas le 1er mauvais sommet)
  Ainsi, v est accessible depuis s.
- Complexité
  - Initialisation : $O(|\text{V}|)$
  - Coût de la boucle tant que :
    - Coût des marquages : $O(|V|)$
    - Coût des insertions et suppressions = 2 × coût des insertions
      En effet, le coût d'une insertion et d'une suppression sont égaux, et chaque sommet inséré doit être supprimé

Chaque sommet est ajouté au sac au plus $\deg_(s)$ fois (ou deg(s) si non-orienté)
Donc au global, le nombre d'insertion est

$$\sum_{s \in V} \deg_(s) = |E| \text{ dans le cas orienté}$$

$$\sum_{s \in V} \deg(s) = 2 \times |E| \text{ cas non-orienté}$$

Finalement, on se retrouve avec un $O(|V| + k(|E|)|E|)$

🔥 ***Remarque***

On peut enrichir WES en adaptant le sac, en calculant des dates de début/fin de traitement, en stockant les prédécesseurs dans le parcours

I.2 Parcours en profondeur décoré

<u>But</u> : Parcourir en profondeur complet un graphe en calculant au passage :

- Les dates de début de de fin de traitement des sommets
- La classification des arcs

```
Parcours(G = (V,E)) =
    P ← pile vide
    D, F ← tableaux de |V| cases contenant des 0 //D servira de marquage
    t ← 0 //Horloge
    explorer (s) =
            Ajouter (-1, s) à P
            Tant que P non vide
                (u, v) ← extraire de P //v est le sommet courant et u son prédécesseur
                Si u = -1
                        t ← t+1
                        D[v] ← t
                        ajouter (v,v) à P
                        pour tout w voisin de v
                                Ajouter (v,w) à P
                Sinon, si u = v
                        t ← t+1
                        F[v] ← t
                Sinon, si D[v] = 0
                        t<t+1
                        D[v] ← t
                        Ajouter (u, v) aux arcs arbres
                        Ajouter (v, v) à P //Pour pouvoir détercter la fin de traitement de v
                        Pour tout w voisin de v
                                Ajouter (v, w) à P
                Sinon //D[v] ≠ 0
                        Si D[v] < D[u]
                            Si F[v] = 0
                                Ajouter (u,v) aux arcs arrières
                            Sinon
                                Ajouter (u,v) aux arcs transverse
                        Sinon
                            Ajouter (u,v) aux arcs avant
    Pour tout s dans V, explorer(s)
```

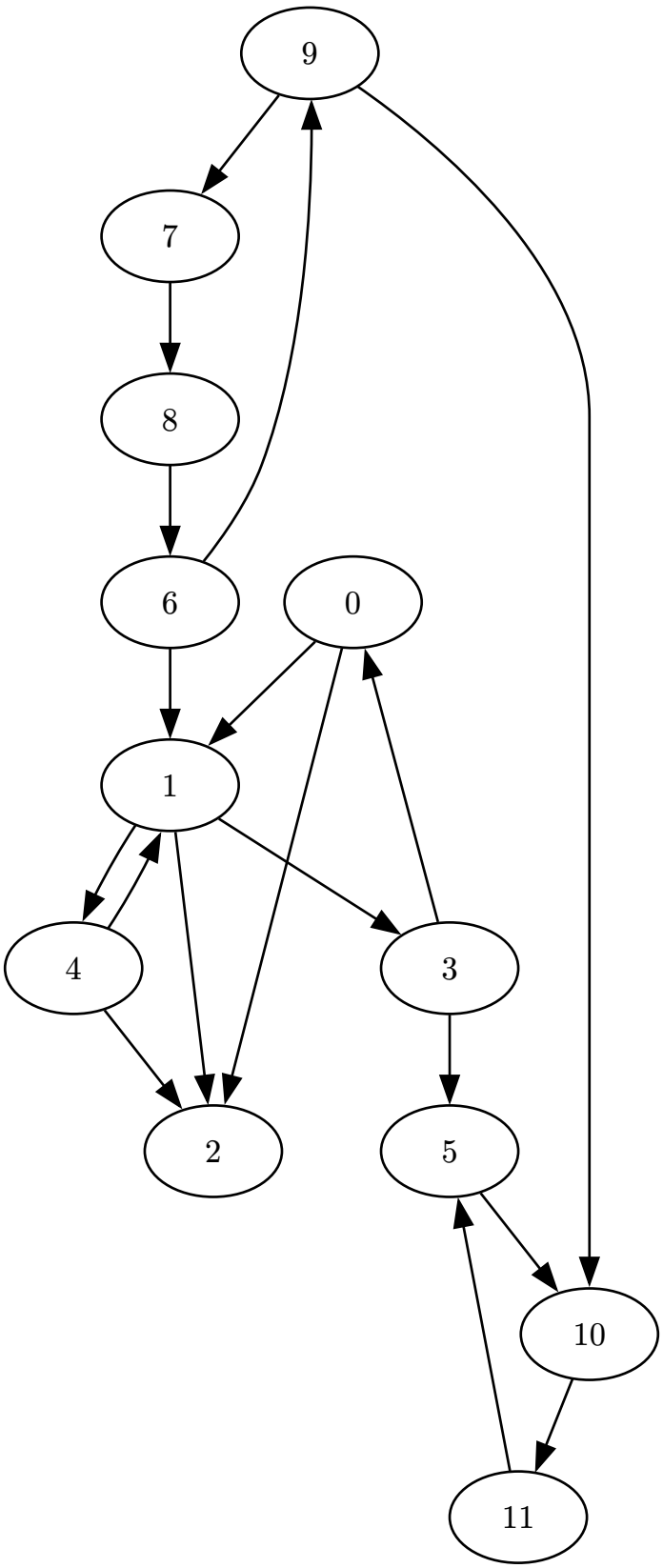


Fig. 1. – Un graphe que l'on va parcourir (oui oui)

Si on a le choix, on explore le sommet de plus petit numéro

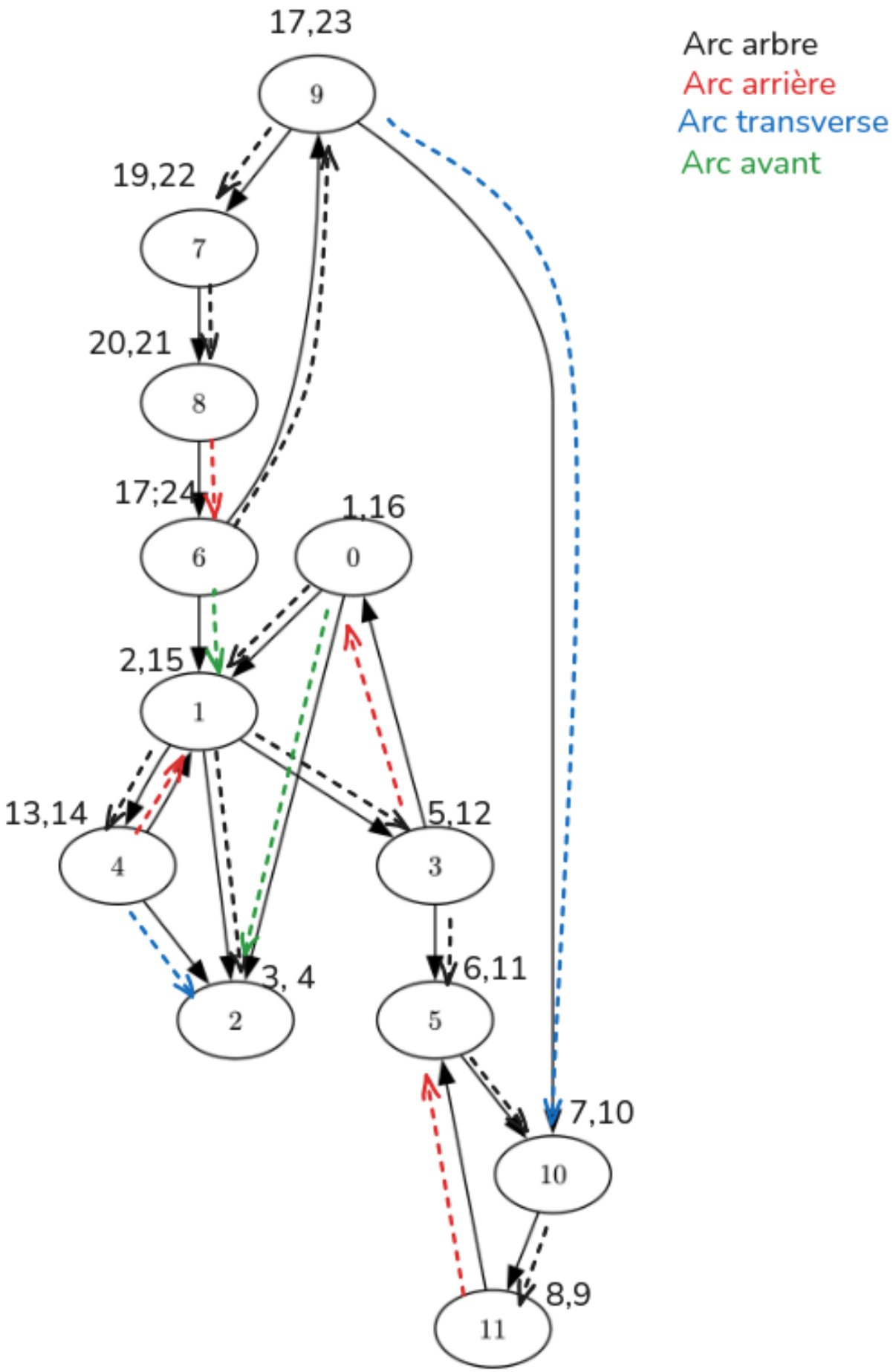


Fig. 2. – Le parcours en question (à refaire au propre)

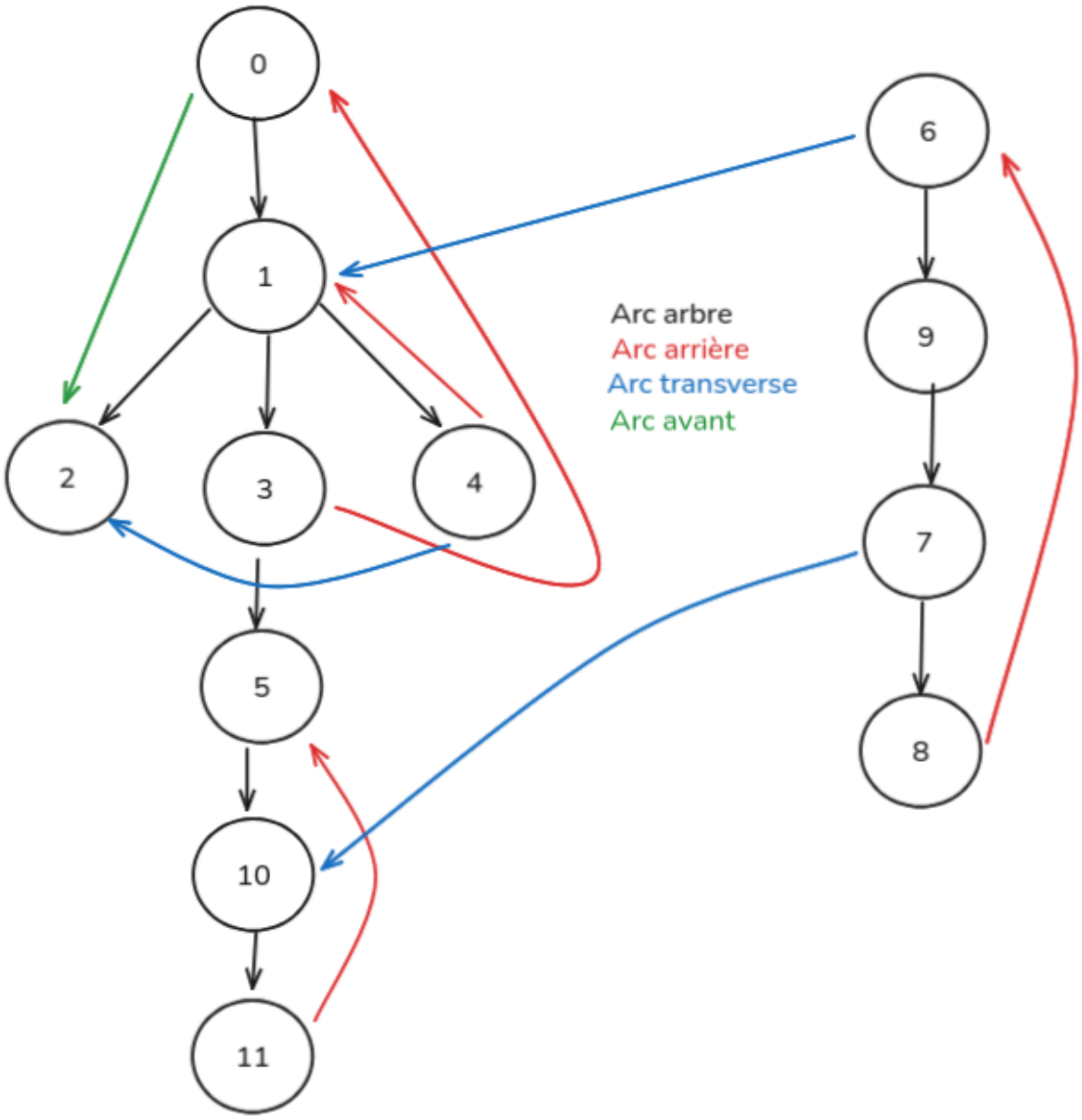


Fig. 3. – Visualisation des types d'arcs sous forme d'arbre

Proposition 2: Si u et v sont deux sommets :
[D[u], F[u]] et [D[v], F[v]] sont soit disjoints soit invlus l'un dant l'autre

<u>Démonstration :</u>

- Supposons $D[u] < D[v]$
  - Si $F[u] < D[v]$, on a disjonction
  - Sinon, $F[u] > D[v]$ et donc v est accessible depuis u
    Donc l'arc fictif (v,v) est mis dans la pile au dessus de l'arc fictif (u,u).
    (v,v) sera donc traité avant (u,u), donc $F[v] < F[u]$. D'où l'inclusion

Définition

- (u,v) est un arc avant si $D[u] < D[v] < F[v] < F[u]$
- (u,v) est un arc arrière si $D[v] < D[u] < F[u] < F[v]$
- (u,v) est un arc transverse si [D[u], F[u]] et [D[v], F[v]] sont disjoints

Remarque

Un arc arbre est donc un arc avant particulier, c'est celui qui permet de découvrir v pour la 1ère fois

II Calcul de composante fortement connexe Contexte : graphes orientés

II.1 Rappels sur les composantes fortements connexes

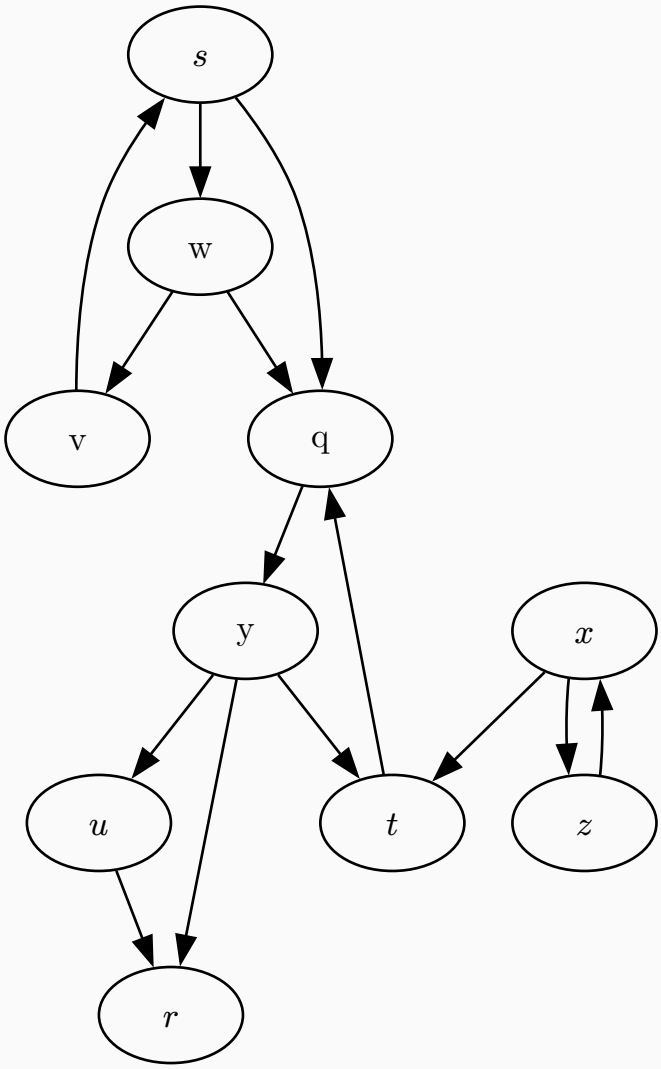
Définition

Une **composante fortement connexe** (CFC) est une classe d'équivalence pour la relation $\rightleftarrows$
$u \rightleftarrows v$ si et seulement si il existe un chemin de u vers v et un chemin de v vers u

Remarque

On confond souvent une CFC avec le sous graphe qu'elle induit

Exemple 1:



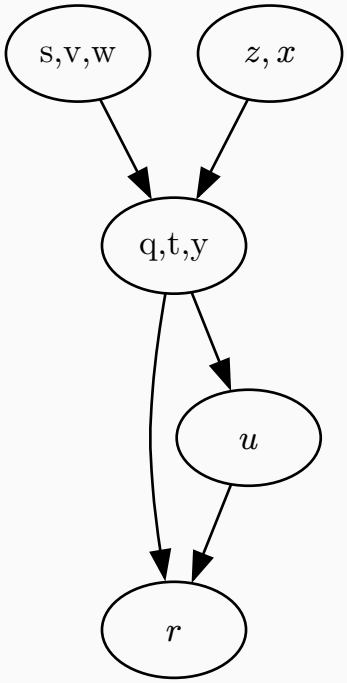
Ici, les CFC sont {s,v,w}, {z,x}, {u}, {r} et {q,t,y}

Définition

Le **graphe des composantes fortements connexes** associé à G est le graphe tel que :

- ses sommets sont les CFC de G
- $c_i \to c_j$ est un arc si et seulement si il existe $x_i \in c_i, x_j \in c_j$ tel que $x_i \to x_j$ dans G

Exemple 2:



Proposition 3: G_{CFC} est acyclique

Démonstation: Supposons pas l'absurde qu'il existe un cycle $c_1 \to c_2 \to ... \to c_k \to c_1$ dans G_{CFC}

Alors, il existe dans le graphe G un chemin $x_1 \to y_2 \rightsquigarrow x_2 \to y_3 \rightsquigarrow x_3 ... y_k \to z_1$ avec $..._i \in c_i$
Alors tous les sommets sur ce chemin sont dans c_1 et les CFC $c_1, c_2, ..., c_k$ sont confondues.

II.2 Algorithme de Kosaraju

On note *parcours_profondeur* un algo de parcours en profondeur qui calcule D et F les dates de début et fin de traitement des sommets et les arbres de parcours dans un parcours complet.

Définition

Si G est un graphe, on note G^T son transposé, c'est-à-dire le graphe G dans lequel on a renversé le sens de tous les arcs

Remarque

Les CFC de G et de G^T sont les mêmes

Algorithme de Kosaraju
Entrée : G un graphe orienté
Sortie : les CFC de G dans un ordre topologique (si $c_1, ..., c_k$ est la liste des CFC renvoyées, si $c_i \to c_j$ alors $i < j$)

```
Lancer parcours_prondeur sur G
Calculer G^T
Lancer parcours_prondeur sur G^T en considérant les sommets par ordre de fin de traitement décroissant selon (1)
Renvoyer les arbres de parcours obtenus en (3) //(dans l'ordre dans lequel ils ont été calculés)
```

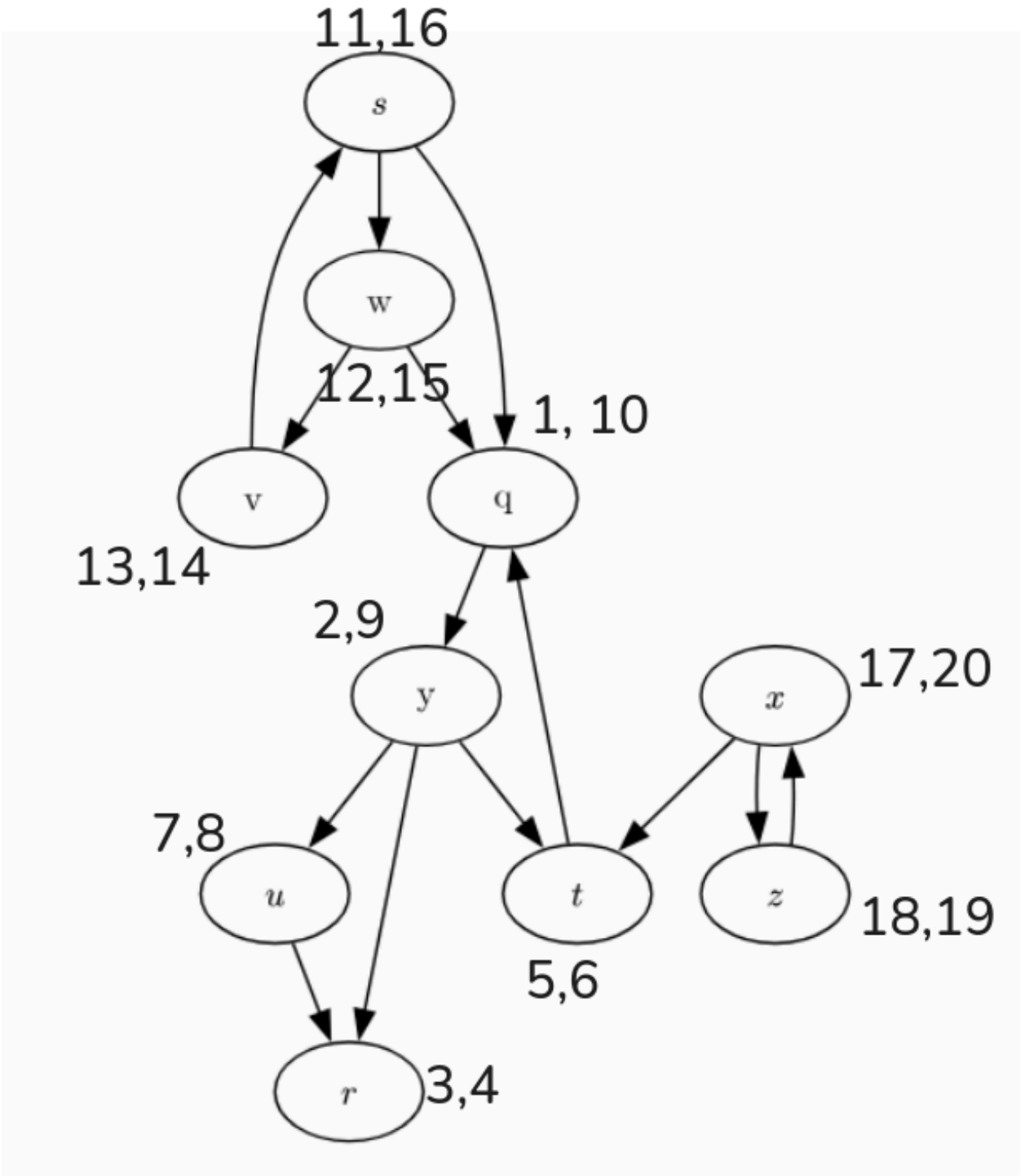
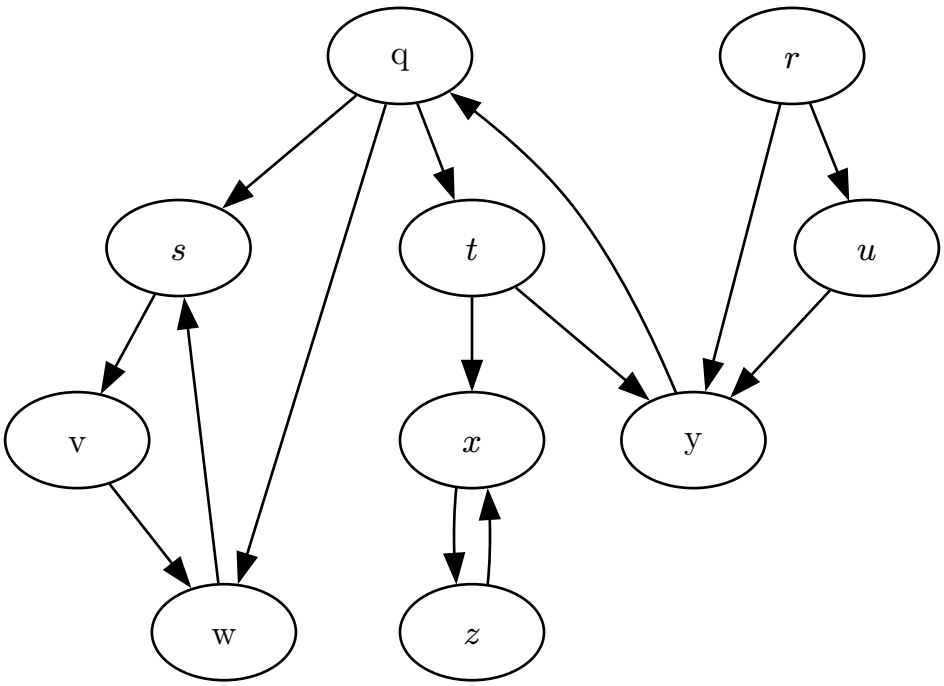


Fig. 6. – Tri topologique du graphe, on obtient [x,z,s,w,v,q,y,u,t,r]



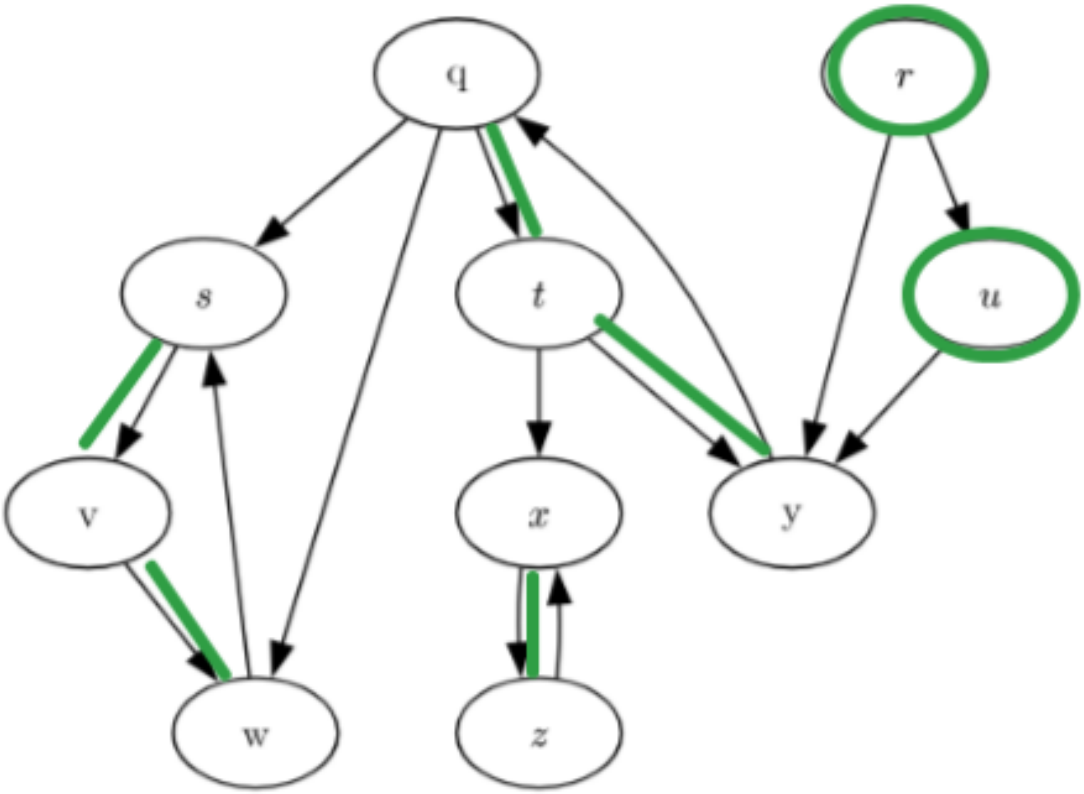


Fig. 8. – Etape 2 à 4, on obtient {x,z}, {x,v,w}, {q,t,y}, {u}, {r}

Proposition 4: L'algorithme de Kosaraju est correct

Lemme:
Pour chaque CFC de G, on note $f(c) = \max_{s \in C} F(s)$ la date de fin totale de traitement de C selon le parcours 1.
Si C $\rightarrow$ C' dans G_{CFC} alors $f(C) > f(C')$
<u>Preuve du lemme</u> : On note x le 1er sommet de $C \cap C'$ parcouru par parcours 1

- 1er cas : $x \in C$
  Comme $C \rightarrow C'$, le parcours de x déclenche le parcours de tous les sommets dans C'
  Donc $\forall y \in C'$, y est terminé d'être parcouru avant que x ne soit terminé d'être parcouru (car parcours en profondeur).
  $\forall y \in C', f(C) = \max_{s \in C} F[s] \geq F[x] > \max_{y \in C'} F[y] = f(C')$
- 2e cas : $x \in C'$
  $f(C') = F[x]$ vu le fonctionnement du parcours en profondeur.
  De plus, le parcours de x ne déclenche aucun parcours d'un sommet $y \in C$, sinon, C et C' seraient confondus.
  Donc le 1er élément de C qui est parcouru l'est après la date $F[x]$. Il est donc fini après cette datte et donc $\forall y \in C, F[y] > F[x]$, puis $f(C) > F[x] = f(C')$

Démonstation: Montrons à présent que pour tout $k \in [\![1, \text{nb de CFC de G}]\!]$, le ke arbre calculé par le parcours 2 est une CFC de G

Considérons le ke arbre construit. Il l'est à partir du sommet x qui n'est pas encore parcouru et dont la date de fin de traitement selon le parcours 1 est maximale.

On note $A_x = \{\text{sommets dans l'arbre parcourus depuis } x\}$
C_x = la CFC qui contient x
Montrons que $A_x = C_x$

- $C_x \subset A_x$
  En effet, si $y \in C_x$, montrons que y n'est pas encore exploré au moment où le parcours sur x est lancé.
  Si y était exploré, comme il existe un chemin $y \rightsquigarrow x$, alors x le serait aussi, ce qui est contradictoire. Donc $y \in A_x$

- $A_x \subset C_x$, par l'absurde, supposons qu'il existe $y \in A_x \setminus C_x$
  Comme $y \in A_x$, il existe un chemin $x \rightsquigarrow y$ dans G^T. De plus, comme $y \notin C_x$, sur ce chemin il y a un premier sommet $s \in C$ avec $C \neq C_x$.

  Dans G_{CFC}, il y a un arc $C \rightarrow C_x$ donc $f(C) > f(C_x)$ d'après le lemme. Avec $f(C) = F[z]$ pour un certain z dans C.
  On a donc $F[z] > F[x]$.
  De plus, z n'est pas parcouru car comme $y \in A_x$, aucun sommet sur le chemin $x \rightsquigarrow y$ n'est parcouru, en particulier pas s. Donc s n'est dans aucun des arbres

de parcours et donc la CFC de s n'est pas parcourue non plus. En partiulier z ne l'est pas. Ainsi, on obtient une contradiction avec la définition de x

Proposition 5: On peut implémenter l'algorithme de Kosaraju avec une complexité en O(|V| + |E|) si l'entrée est G = (V, E)

Démonstation:
1. Peut se faire en O(|V| + |E|) (parcours)
2. Peut se faire en O(|E|)
3. Peut se faire en O(|V| + |E|)

Total = O(|V| + |E|)

III Arbres couvrant minimaux (Contexte : graphes non orientés et pondérés)

III.1 Arbres couvrants

Définition

Un **arbre** est un graphe acyclique et connexes

Une **forêt** est un graphe acyclique

Définition

Si G = (V, E) est un graphe alors un **arbre couvrant** de G est un graphe (V, A) avec A ∈ E et qui est un arbre

Remarque

On confond souvent le graphe (V, A) et A

Proposition 6: G est connexe si et seulement si il admet un arbre couvrant

Démonstation: cf. cours de sup

Démonstation: (⇒) Via un algorithme par suppression d'arrête
Entrée : G = (V, E) connexe
Sortie : un arbre couvrant de G

```
A ← E
Pour chaque arrête (x,y) de E
    Si retirer (x,y) conserve la connexité de (V,A):
        A ← A\{x,y}
Renvoyer(V,A)
```

Démonstation: (⇒) Via un algorithme par ajout d'arêtes
Entrée : G = (V, E) connexe
Sortie : un arbre couvrant de G

```
A ← vide
Pour chaque arête (x,y) de E
    Si ajouter (x,y) à A ne crée pas de cycle dans (V,A)
        A ← A+{(x,y)}
Renvoyer (V,a)
```

⚠️ **Notation**

Dans la suite, p est la fonction de pondération des arêtes.

Définition

Le poids d'un sous-graphe H de G est noté p(H) et est

$$\sum_{(x,y) \text{ arête de H}} (\mathrm{p}(x,y))$$

Définition

Un **arbre couvrant minimal** (ACM) de G est un arbre couvrant H de G tel que p(H) = min{p(A) | A arbre couvrant de G}

🔥 ***Remarque***

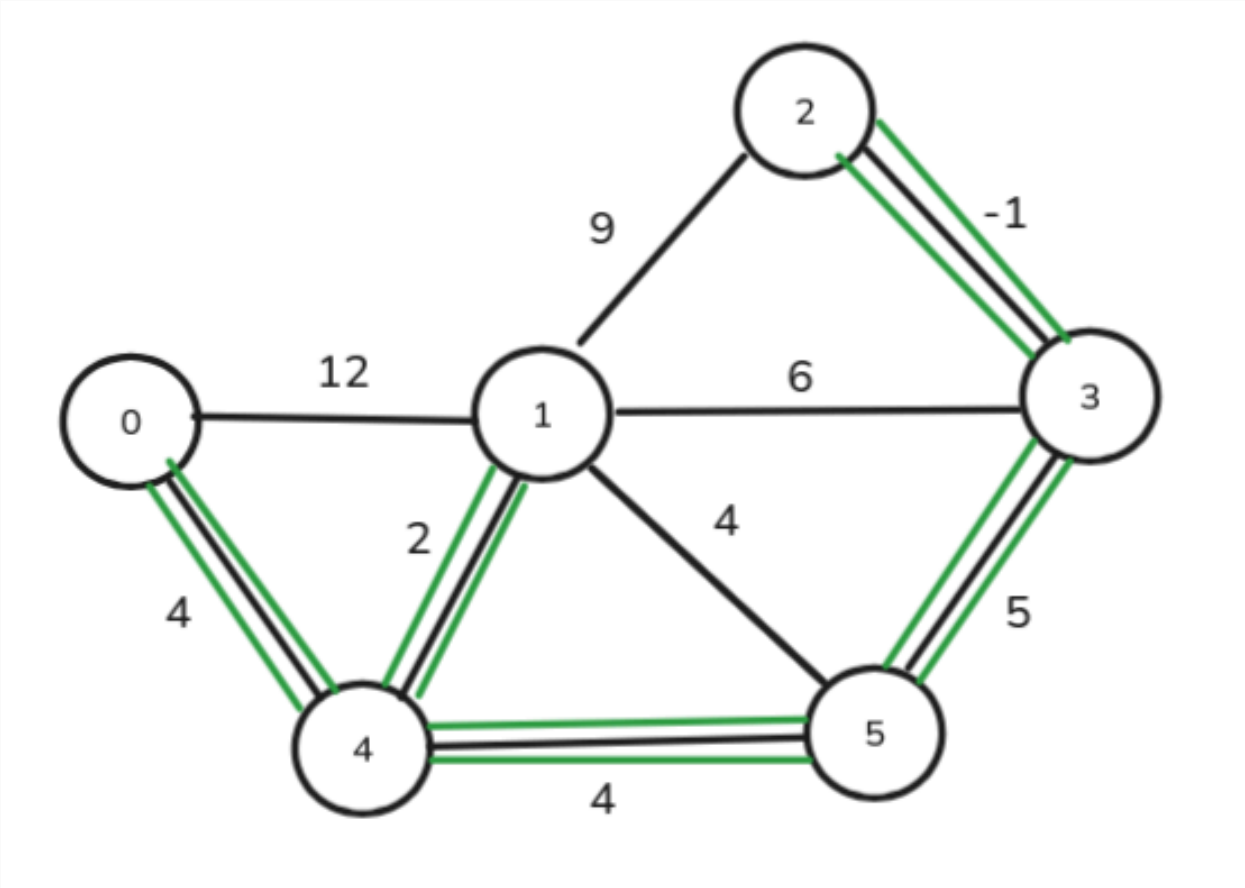
Si G est connexe, il admet nécessairemment un ACM (pas forcément unique)

III.2 Algorithme de Kruskal

Algorithme de Kruskal
Entrée : G = (V, E) non orienté pondérés connexe
Sortie : Un ACM de G

```
T ← vide
Pour chaque arête (x,y) prise dans l'ordre croissant des poids
        Si ajouter (x,y) à (V, T) le laisse acyclique
              T ← T+{(x,y)}
Renvoyer (V, T)
```

Exemple 3:



Proposition 7: L'algorithme de Kruskal esr correct

Démonstation: On note A le graphe renvoyé par cet algo

- **A est un arbre couvrant de G** car l'algorithme de Kruskal estu n cas particulier de construction d'arbre couvrant par ajout d'arêtes
- **Supposons par l'absurde que A n'est pas minimal**
  Notons A^* un arbre couvrant minimal de G qui partage avec A le plus d'arêtes possibles.

  Notons $e_0, ..., e_{|V|-2}$ les arêtes de A dans l'ordre dans lequel elles sont sélectionnés par Kruskal.
  Notons e_i la 1ère de ces arêtes qui n'est pas dans A^* Alors $A^* \cup \{e_i\}$ contient un cycle. Sur ce cycle, il existe $e \notin \{e_1, ..., e_{i-1}\}$, sinon il y aurait un cycle dans l'arbre A.
  Considérons alors $A' = (A^* \setminus \{e\}) \cup \{e_i\}$, c'est un arbre couvrant de G.
  Comme A^* est un ACM : $p(A^*) \leq p(A') = p(A^*) - p(e) + p(e_i)$ donc $p(e) \leq p(e_i)$

→ **Si $\mathbf{p(e) < p(e_i)}$** alors e aurait été considérée avant e_i par l'algorithme, et au moment où e est considérée, l'ajouter ne crée pas de cycle, sinon il y en aurait dans A*. Donc on aurait du ajouter e à A, ce qui est contradictoire.

→ **Sinon, $\mathbf{p(e) = p(e_i)}$** et A' est un ACM qui a en commun avec A les arêtes $\{e_0, e_1, ..., e_i\}$, soit une de plus que A*, ce qui en contredit la définition.

Ainsi, p(A) est minimal

III.3 Structure union-find

Le coût pour Kruskal est la somme du :
- Coût du tri : O(|E|log|E|)
- Coût de tester si x et y sont dans une même composante connexe de (V, T)

Définition

Une **structure union-find** est une structure de donnée permettant de gérer une partition d'un ensemble X via les fonctions d'interface suivante :
- *creer_partition*(X) crée une partition de X en singletons
- *trouver*(P, $x \in X$) renvoie un représentant canonique c de la partie qui contient x
  Si x et y sont dans la même partie de P, *trouver*(P, x) = *trouver*(P, y)
- *unir*(P, x, y) remplace les partie P_x et P_y contenant respectivement x et y par $P_x \cup P_y$. Cette fonction détermine au passage le nouveau représentant de P_x et P_y

Si on dispose d'une telle structure, l'algorithme de Kruskal se réécrit :

```
T ← vide
P ← creer_partition(V)
Pour chaque arête (x,y) de E par ordre croissant des poids
        Si trouver(P,x) ≠ trouver(P,y)
                T ← T+{(x,y)}
                unir(P,x,y)
Renvoyer(V,T)
```

Coût hors tri =
$1 \times$ coût de *creer_partition*
$+ 2|E| \times$ coût de *trouver*
$+ (|V| - 1) \times$ coût de *unir*

? Question

Comment implément une structure union-find en pratique ?

Dans la suite on numérote à partir de 0 les élément de X pour manipuler une partition de $[\![0, n-1]\!]$ avec $n = |X|$

III.3.a Implémentation naïve

On stoche dans un tableau de taille n, en case i : le plus petit numéro d'un élément dnas le même morceau que i

Exemple 4:

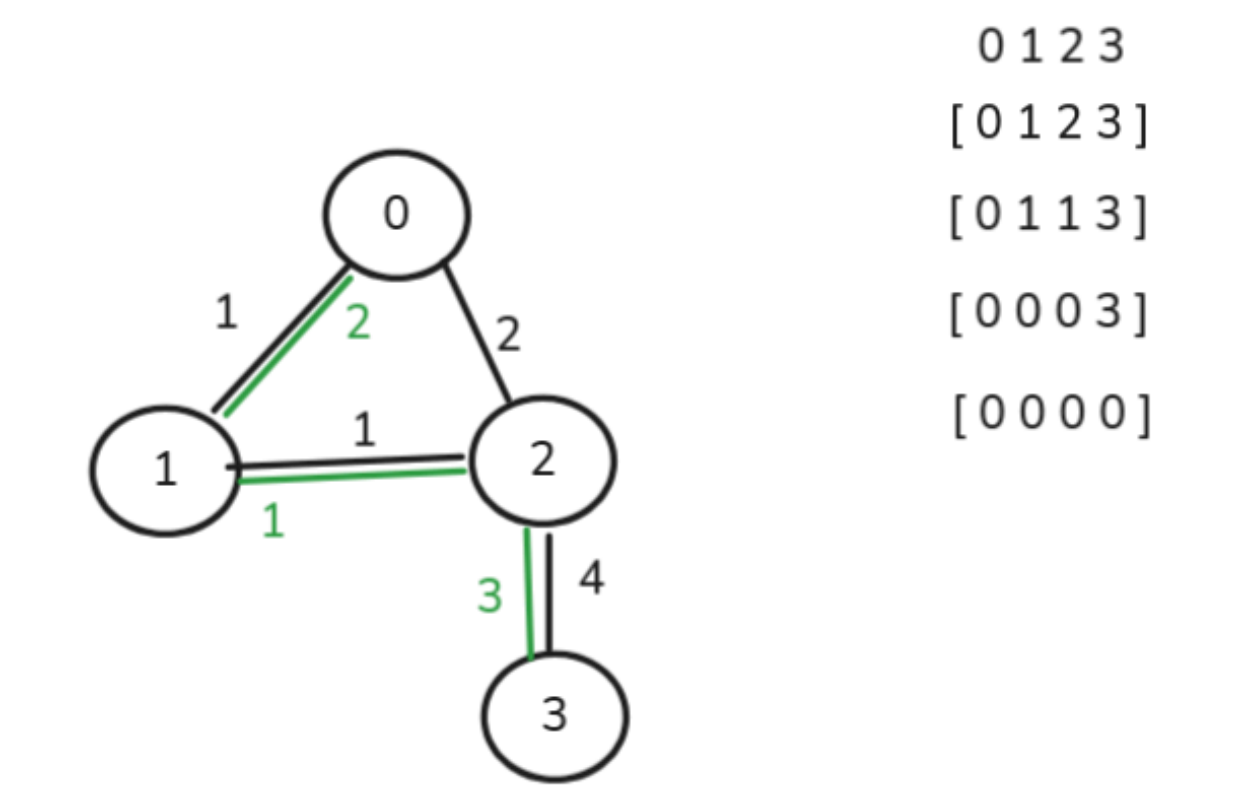


Fig. 10. – Un exemple de l'algorithme de Kruskal avec l'implémentation naïve

Avec cette implémentation :

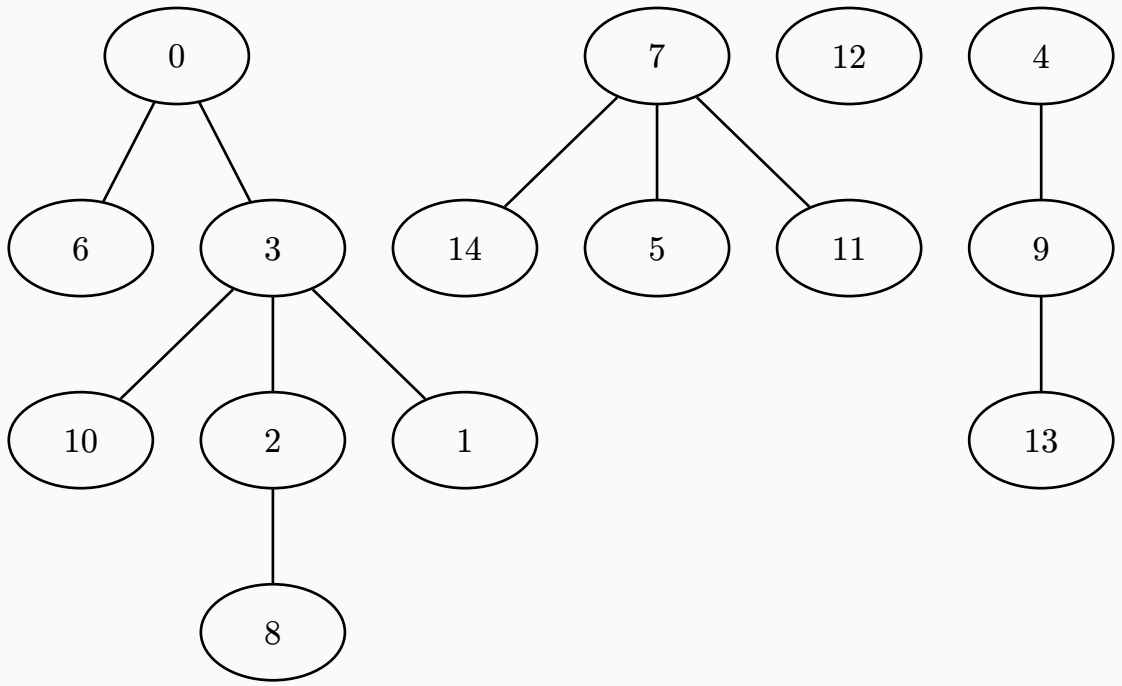
- Une partition se fait en $O(|X|)$
- Trouver en $O(1)$
- Unir en $O(|X|)$ car l'union consiste à remplacer un des représentants par l'autre ce qui nécessite de parcourir le tableau.

Avec cette implémentation, coût de Kruskal hors tri $= O(|V| + |E| + |V|^2) = O(|V|^2)$
En comptant le tri $= O(|V|^2 + |E|\log|E|)$

III.3.b Implémentation arborescente

Idée : chaque morceau de X est un arbre dont le représentant est la racine

Exemple 5:



est une représentation de la partition {0,3,6,10,2,1,8}⊔{7,14,5,11}⊔{12}⊔{4,9,13} de $[\![6, 14]\!]$

Ce type d'objet peut être représenté par un tableau de taille |X| contenant en case i le père de l'élement i dans la forêt
Par exemple ici on a [−1,3,3,0,-1,7,0,-1,2,4,3,7,-1,9,7]

Avec cette implémentation :

- *creer_partition* coûte O(|X|)
- *trouver* coûte O(|X|)
- *unir* coûte O(|X|) car unir demande à faire 2 appels à trouver puis à connecter les racines ainsi trouvées

III.3.c Implémentation arborescente avec union par hauteur

Idée : pareil avec un tableau supplémentaitre qui stocke en case i la hauteur du sous-arbre enraciné à i (mis à jour à chaque union en temps constant), de plus, quand on fait une union, on récupère les hauteurs h_1 et h_2 des deux arbres de racines respectives r_1 et r_2

- Si $h_1 < h_2$, r_2 devient la racine globale
- Si $h_1 > h_2$, r_1 devient la racine globale
- Si $h_1 = h_2$, au hasard parmi r_1 et r_2

Proposition 8: En utilisant une union par hauteur, les complexités de unir et trouver sont en O(log|X|)

Démonstation:
Montrons q'une partie qui contient p élément est représentée par un arbre de hauteur inférieure à $\log_2 \mathrm{p}$

<u>Initialement</u>, c'est vrai car chaque partie contient un élément et est représenté par un arbre de hauteur 0 (car réduit à une racine).

<u>Si on avait cette propriété</u>, et qu'on unit deux parties de cardinal p_1 et p_2 et dont les arbres représentatifs ont hauteur h_1 et h_2

- Si $h_1 \neq h_2$, on peut supposer $h_1 > h_2$ et alors, la partie résultant de l'union est représentée par un arbre qui contient $\mathrm{p} = \mathrm{p}_1 + \mathrm{p}_2$ élément et est de hauteur h_1.
  Or, par hypothèse, $h_1 \leq \log_2 \mathrm{p}_1 \leq \log_2(\mathrm{p}_1 + \mathrm{p}_2)$
- Si $h_1 = h_2$ alors la partie résultant de l'union est représentée par un arbre qui contient $\mathrm{p} = \mathrm{p}_1 + \mathrm{p}_2$ élément et est de hauteur $h_1 + 1$.
  Or, on sait que $h_1 < \log_2 \mathrm{p}_1$ et $h_2 < \log_2 \mathrm{p}_2$
  Donc $\mathrm{p} = \mathrm{p}_1 + \mathrm{p}_2 \geq 2^{h_1} + 2^{h_2} = 2^{h_1+1} = 2^h$, c'est-à-dire $h \leq \log_2 \mathrm{p}$

La compleixté de *trouver*(P, x) est en O(h) avec h la hauteur de l'arbre qui contient x donc en O(log|A_x|) avec |A_x| le nombre d'élément dans l'arbre de x, donc en O(log|X|)
unir consiste à appeler deux fois *trouver* et à des opérations en temps constant, donc est en O(log|X|) aussi

Avec cette implémentation :

Proposition 9: L'algorithme de Kruskal peut être implémenté avec une complexité temporelle en O(|E|log|V|)

Démonstation:
Coût du tri = O(|E|log|E|) = O(|E|log|V|) car log|E| $\leq$ 2log|V|
Coût de *creer_partition* = O(|V|)
Coût de *trouver* = O(|E|log|V|)
Coût de *unir* = O(|V|log|V|)

Et avec |E| > |V| car G est connexe

III.3.d Implémentation arborescente avec compression de chemin et union par rang

Exemple 6:

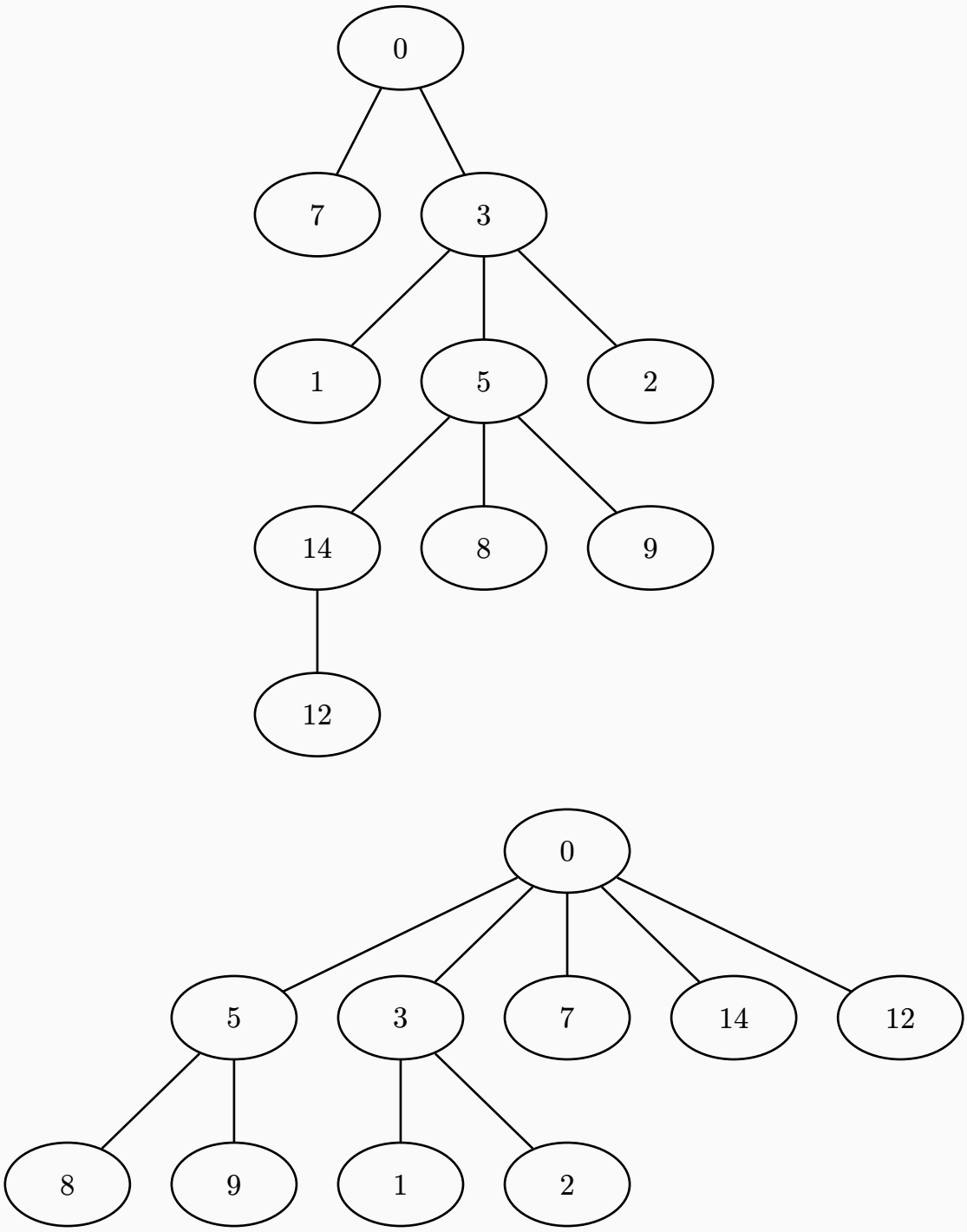


Fig. 12. – Un exemple de compression sur un arbre

Idée : On comprime les chemin et on conserve l'union par hauteur en la renommant union par rang

Proposition 10: La complexitée amortie de unir et trouver si la taille de l'ensemble partitionné est n est $\mathrm{O}(\alpha(n))$
Où α est la réciproque de la fonction d'Ackermann. *(Pour n'importe quelle application pratique, $\alpha(n) \leq 5$)*

IV Calcul du plus court chemin à origine et destination unique

IV.1 Rappels sur l'algorithme de Dijksta

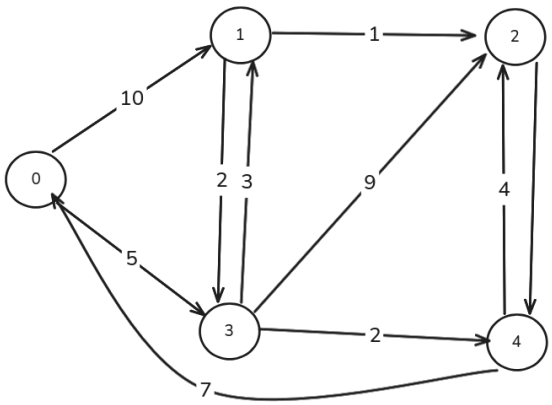
Cadre : On travaille dans des graphes orientés ou non, et pondérés par une fonction de pondération p.
On note p(e) le poirds de l'arête e, p(c) le poids du chemin c

$$p(c) = \sum_{\text{e_i sur C}} p(e_i)$$

> ***i*** **Définition**
>
> Si G est pondéré de sorte à ce qu'il ne contienne aucun cycle de poids strictement négatif
> - **la distance de x à y**, notée δ(x,y) est
>
> δ(x,y) = inf{p(c) | c est un chemin de x vers y}, avec inf($\emptyset$) = $+\infty$
>
> - Un **plus court chemin de x vers y** est un chemin de x vers y de poids δ(x,y)

Algorithme de Dijksta (marquage en entrée de sac)
<u>Entrée</u> : Un graphe G = (V,E) pondéré par une fonction de pondération positive, et s un sommet
<u>Sortie</u> : Les poids des plus court chemins à origine s dans G / Ces plus court chemins directement

```
D ← tableau de |V| cases contenant +infty //Sert aussi de marquage
F ← file de priorité mine vide
Insérer s dans F avec priorité 0
Tant que F non vide :
        (x, dx) ← extraire de F
        Si D[x] = +infty
                D[x] ← dx
                Pour tout voisin y de x :
                        Insérer y dans F avec priorité D[x] + p(x,y)
Renvoyer D
```



Contenu de F (sommet, priorité)	D[0]	D[1]	D[2]	D[3]	D[4]
(0,0)	inf	inf	inf	inf	inf
(1,10) ; (3,5)	0	inf	inf	inf	inf
(1,10) ; (1,8) ; (2,14) ; (4,7)	0	inf	inf	5	inf
(1,10) ; (1,8) ; (2,14) ; (0,14) ; (2,11)	0	inf	inf	5	7
(1,10) ; (2,14) ; (0,14) ; (2,11) ; (3,10) ; (2,9)	0	8	inf	5	7
(1,10) ; (2,14) ; (0,14) ; (2,11) ; (3,10) ; (4,15)	0	8	9	5	7
Puis la file se vide					

Fig. 13. – Un exemple d'application à un graphe orienté

Proposition 11: L'algorithme de Dijkstra appliqué à G = (V,E) a une complexité temporelle pire cas en O(|V| + |E|log|V|) si G est représenté par listes d'adjacence

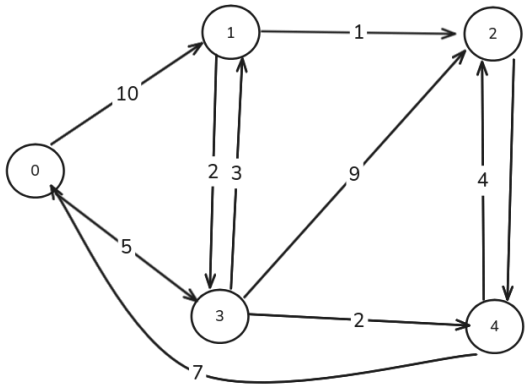
Démonstation: $\log(|E|) \leq \log(|V|^2) = 2\log|V|$
Il s'agit d'un parcours classique avec une file de priorité comme sac

Algorithme de Dijksta (marquage en entrée de sac)
Les lignes comportant un (*) permettent de calculer les plus court chemins explicitement

```
1       D ← tableau de |V| cases contenant +infty
2   (*) P ← tableau de |V| cases conetnant -1
3       D[s] ← 0
4       F ← file de priorité min vide
5       Insérer s dans F avec priorité 0
6       Tant que F non vide :
7               (x, dx) ← extraire de F
8               Pour tout y voisin de x
9                       d ← D[x] + p(x,y)
10                      Si d < D[y] alors
11                              D[y] ← d
12  (*)                         P[y] ← x
13                              inserer_ou_diminuer(F,y,d)
14      Renvoyer D
15
16      inserer_ou_diminuer :
17              Si y n'est pas dans E, l'insérer avec priorité D, sinon,
                diminuer la priorité de y pour qu'elle valle d
```

🏁 **Exercice**

Implémenter *inserer_ou_diminer* en O(log|E|)



Contenu de F (sommet, priorité)
(0,0)
(1,10) ; (3,5)
(1,8) ; (2,14) ; (4,7)
(1,8) ; (2,11)
(2,9)
Fini

D[0]	D[1]	D[2]	D[3]	D[4]
0	inf	inf	inf	inf
0	10	inf	5	inf
0	8	14	5	7
0	8	11	5	7
0	8	9	5	7

Fig. 14. – Le même exemple sur la deuxième version de l'algorithme

🔥 ***Remarque***

Les deux versions de l'algorithme calculent toutes les deux des plus court chemins et ont la même complexité temporelle en O(|V| + |E|log|V|)

Pour la version avec marquage précoce, la complexité spatiale est meilleure car la file a une taille en O(|V|) plutôt qu'en O(|E|)
Pour la version avec marquage tardif, elle est plus facile à analyser, et il n'y a pas besoin d'implémenter la diminution de priorité

Cet algorithme est très bien pour trouver tous les plus court chemins, mais quand il y a une destination précise à trouver il n'est pas du tout optimal car il cherche de tous les côtés en même temps dans une sorte de sphère qui grandit. Une idée pour l'améliorer est de faire en sorte que l'exploration soit dirigée vers le but plutôt qu'elle rayonne indistinctement depuis l'origine.

Algorithme basé sur Dijkstra pour calculer un plus court chemin à source s et but b

```
D ← tableau de |V| cases contenant +infty
P ← tableau de |V| cases contenant une valeur par défaut
F ← file de priorité min vide
D[s] ← 0
Insérer s dans F avec priorité 0
Tant que F non vide :
        (x, dx) ← extraire de F
        Si x = b, return reconstruire_chemin(b, P)
        Pour tout y voisin de x
                d ← D[x] + p(x,y)
                Si d ⩽ D[y] alors
```

```
                        D[y] ← d
                        P[y] ← x
                        inserer_ou_diminuer(F,y,d)
Renvoyer une valeur signifiant une absence de chemin

inserer_ou_diminuer(F, y, d) :
        Si y n'est pas dans F
                l'insérer avec priorité d
        Sinon
                diminuer la priorité de y pour qu'elle valle d

reconstruire_chemin(b, P) :
        Reconstruit le chemin de b vers s en remontant le tableau P
        jusqu'à tomber sur la valeur par défaut
        Retourne le chemin pour obtenir celui de s vers b
```

IV.2 Algorithme A*

Le principe d'A* est d'utiliser une heuristique h pour orienter la recherche du but. On veut que h(x) renvoie une estimation de la distance entre x et le but. On modifie alors la priorité des sommets de (distance source → x) à
(distance source → x) + (distance estimée x → but)

En supposant que cette heuristique existe, l'algorithme A* est :

```
D ← tableau de |V| cases contenant +infty
P ← tableau de |V| cases contenant une valeur par défaut
F ← file de priorité min vide
D[s] ← 0
Insérer s dans F avec priorité h(s)
Tant que F non vide :
        (x, dx) ← extraire de F
        Si x = b, return reconstruire_chemin(b, P)
        Pour tout y voisin de x
                d ← D[x] + p(x,y)
                Si d ⩽ D[y] alors
                        D[y] ← d
                        P[y] ← x
                        inserer_ou_diminuer(F, y, d + h(y))
Renvoyer une valeur signifiant une absence de chemin
```

🔥 ***Remarque***

En pratique, on utilise un dictionnaire comme structure de marquage (pour D et P), afin d'éviter d'avoir des tableaux de très très grande taille dont seules quelques cases seront utilisées.

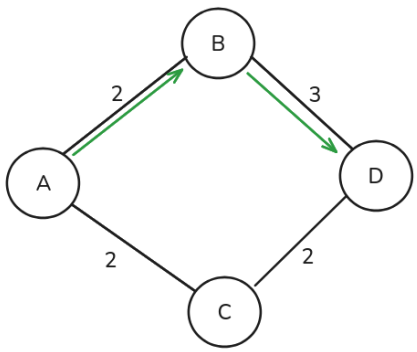
🔥 ***Remarque***

Si h = 0, alors A* n'est autre que Dijksta

? Question

Est-ce que A* calcule bien un plus court chemin de la source vers le but ?
Non en général

Exemple 7:



h(A) = 3
h(B) = 2
h(C) = 5
h(D) = 0

File F	D[A]	D[B]	D[C]	D[D]
(A,3)	0	inf	inf	inf
(B,4) ; (C,7)	0	2	2	inf
(C,7) ; (D,5)	0	2	2	5

Fig. 15. – Si l'heuristique est mal choisie, A* n'est pas correcte

IV.3 Heuristiques admissibles et monotones

Contexte : On veut calculer un plus court chemin dans G = (V,E) pondéré avec des pondérations toutes positives, depuis un sommet s et vers un sommet b

i Définition

Une heuristique h est
- **admissible** si pour tout sommet x, h(x) $\leq \delta$(x,b)
- **monotone**, ou **cohérente** si pour toute arête (u,v), h(u) $\leq$ h(v) + p(u,v)

Proposition 12: Si h est monotone et h(b) = 0, alors h est admissible.

Démonstation: Montrons par récurrence sur $n \in \mathbb{N}$ la propriété
$H(n)$: Pour tout sommet v tel que la longueur d'un plus court chemin (en terme de poirds) entre v et b vaut n, on a h(v) $\leq \delta$(v,b)

Si v est à distance 0 de b en terme de nombres d'arcs, on a v = b.
On a bien h(b) = 0 = δ(b,b)

Soit $n \in \mathbb{N}$ tel que $H(n)$. Soit v un sommet tel qu'un plus court chemin de v à b a n+1 arcs. Notons v = $u_0 \to u_1 \to ... \to u_{n+1}$ avec u_{n+1} = b.
Alors $u_0 \to u_1$ est un chemin de poids p(u_0, u_1), et $u_1 \to ... \to u_{n+1}$ est un plus court chemin en terme de poids de n arcs de u_1 à b.
On peut donc lui appliquer l'hypothèse de récurrence : h$(u_1) \leq \delta(u_1$,b) (*)

$$\begin{aligned}
\text{Or, h}(v) = \text{h}(u_0) &\leq \text{h}(u_1) + \text{p}(u_0, u_1) \text{ par monotonie de h} \\
&\leq \delta(u_1, \text{b}) + \text{p}(u_0, u_1) \text{ par (*)} \\
&= \delta(u_1, \text{b}) + \delta(u_0, u_1) \text{ car } \ u_0 \to u_1 \text{ est un plus court chemin} \\
&\text{en tant que sous chemin d'un plus court chemin} \\
&= \delta(\text{v,b})
\end{aligned}$$

On ainsi montré que pour tout sommet v tel qu'il existe un chemin de v vers b, on a h(v) $\leq \delta$(v,b).
Pour les sommets v tels qu'il n'xiste pas de chemin de v vers b, h(v) $\leq \delta$(v,b) car δ(v,b) = $+\infty$

Exemple 8: Dans l'exemple précédant, l'heuristique n'est pas admissble car h(C) > δ(C,D). De plus h(D) = 0 donc par contraposé de la proposition précédente alors h n'est pas monotone (par exemple, h(C) > h(D) + p(C,D) car 5 > 2)

🔥 ***Remarque***

Si h est monotone est nulle au but, elle est admissible mais la réciproque est fausse.

Exemple 9:

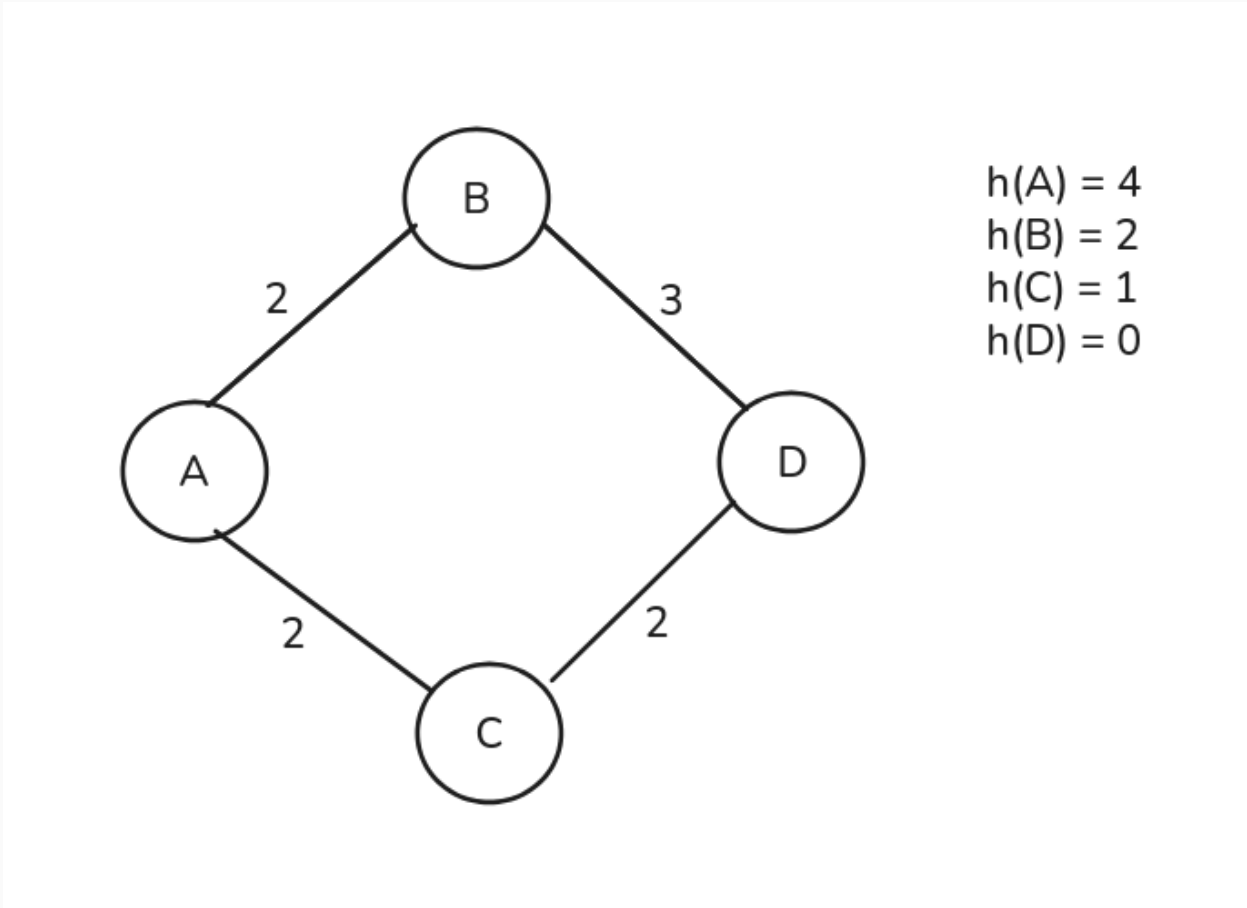


Fig. 16. – Ici, l'heuristique est admissible mais non monotone.
Par exemple, h(A) = 4 > 3 = h(C) + p(A,C)

Proposition 13: Si h est admissible, alors A* renvoie un plus court chemin de s vers b

Démonstation: S'il n'y a pas de chemin de s vers b dans G, alors comme A* est un cas particulier de parcours, il renverra $\emptyset$, ce qui est bien le résultat attendu.

Sinon, il existe un chemin, donc un plus court chemin de s vers b.
En tant que parcours, A_* renvoie bien un chemin C de s vers b. Il reste donc à montrer qu'il a un poids minimal.
L'idée générale est de supposer par l'absurde que C n'est pas minimal. On se saisit alors d'un chemin C' qui est un plus court chemin (et donc strictement plus court). On montre alors que tous les sommets sur C' sont extraits de la file dans A* *au moins une fois strictement avant b. En particulier, comme b est sur le chemin C', alors il aura été extrait au moins une fois strictement avant lui-même, ce qui est impossible.*

<u>Étape 1</u> : On note d le poids de C et d' le poids de C' [d' < d].
On montre que la priorité de b au moment où on l'extrait de F est d.

La priorité de b au moment de l'extraction est D[b] + h(b). Or, h(b) $\leq \delta$(b,b) = b. Et h(b) ≥ 0 car h(b) est l'estimation d'une distance dans un graphe pondéré positivement. Donc h(b) = 0.
Ainsi, D[b] + h(b) = D[b] = d.

<u>Étape 2</u> : On note $(u_0, ..., u_n)$ les sommets sur C'
On veut montrer que pour tout i dans $[\![1, n]\!]$ que u_i a eu une priorité strictement inférieur à d au cours de l'exécution de A*

Pour ça, on montre par récurrence sur $i \in [\![1, n]\!]$
$H(n)$: u_i a été extrait de F avec une priorité strictement inférieur à d au cours de A* et au moment de cette extraction, D$[u_i] \leq \delta(u_0, u_i)$

Initialisation : Le fonctionnement de A* indique que u_1 a été mis dans la file suite au traitement de u_0 avec une priorité D$[u_0]$ + h(u_1) = h(u_1)
et on a aussi fixé D$[u_1]$ à D$[u_0]$+p(u_0, u_1) = p(u_0, u_1)= $\delta(u_0, u_1)$ car (u_0, u_1) est un sous chemin d'un plus court chemin donc un plus court chemin.

Comme h est admissible,

$$\begin{aligned}
\text{La priorité de } u_1 &= \mathrm{D}[u_1] + \mathrm{h}(u_1) \\
&\leq \mathrm{p}(u_0, u_1) + \delta(u_0, b) \text{ par admissibilité de h et} \\
&\text{par le fonctionnement de A*} \\
&= \delta(u_0, b) = d' \text{ car } \mathrm{p}(u_0, u_1) + \delta(u_1, b) \text{ est le poids de C'} \\
&< d \text{ car C n'est pas minimal}
\end{aligned}$$

Comme la priorité de u_1 et D$[u_1]$ ne peuvent que diminuer, u_1 vérifie bien e qu'on veut.

Hérédité : Soit $i \in [\![1, n]\!]$, on suppose $H(i)$.
u_1 est extrait de F au moins une fois strictement avant b, donc déclenche l'ajout de u_{i+1} à F (car passer par u_i améliore le chemin vers u_{i+1} puisque c'est un plus court chemin).

À ce moment, d'après A*,

$$\begin{aligned}
D[u_{i+1}] = D[u_i] + \mathrm{p}(u_i, u_{i+1}) &\leq \delta(u_0, u_i) + \mathrm{p}(u_i, u_{i+1}) \\
&\leq \delta(u_0, u_{i+1})
\end{aligned}$$

$$\begin{aligned}
\text{Priorité de } u_{i+1} &= D[u_{i+1}] + h(u_{i+1}) \\
&\leq \delta(u_0, u_{i+1}) + \delta(u_{i+1}, b) \text{ par admissibilité de h} \\
&\leq d' \\
&< d
\end{aligned}$$

D'où le but

Proposition 14:

- Si h est monotone, la complexité de A* est la même que celle de Dijksta , donc en O(|V| + |E|log|V|)
- Si h est uniquement admissible, la complexité de A* peut être exponentielle en la taille du plus court chemin de s à b car un même sommet peut être extrait de la file de priorité avec des distances estimées à s de plus en plus petites.

Exemple 10:

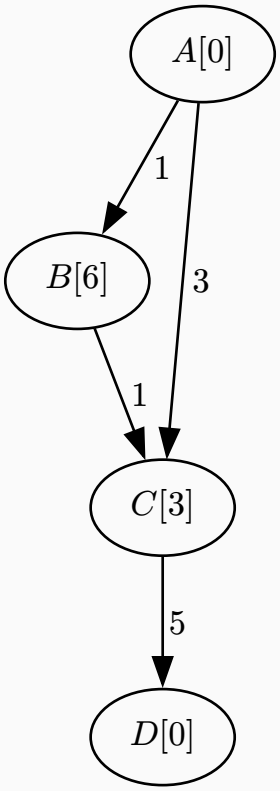


Fig. 17. – Un exemple de complexité exponentielle

On veut un plus court chemin de A à D.
h est admissible mais non monotone car $h(B) > h(C) + (B, C)$

Contenu de F	D[A]	D[B]	D[C]	D[D]
(A,0)	0	$+\infty$	$+\infty$	$+\infty$
(C,6), (B,7)	0	1	3	$+\infty$
(B,7), (D,8)	0	1	3	8
(C,5), (D,8)	0	1	2	8
(D,7)	0	1	2	7

On voit ici que C est inséré et supprimé plusieurs fois, ici cela n'arrive qu'une fois mais sur des graphes plus conséquents ce phénomène peut se répéter, ce qui entrainera en plus l'actualisation de tous les enfants du sommet concernés à chaque fois, ce qui entraîne le coût exponentiel.

V Couplages dans les graphes biparis

(Contexte : graphes non orientés)

V.1 Problème de couplage

Définition

Soit G = (V,E) un graphe orienté M est un **couplage** de G si $M \subset E$ tel qu'on n'a jamais deux arêtes de M incidente à un même sommet de G.

Un sommet qui est une extrémité d'une arête de M est dit **saturé** ou **couvert** (par M).

Un sommet qui n'est pas saturé est **libre** ou **exposé**.

Définition

Un couplage est dit
- **maximal** (pour l'inclusion) s'il n'existe aucune pair de sommet adjacents non saturés par M.
- **maximum** (pour le cardinal) s'il n'existe pas de couplages contenant plus d'arêtes.
- **parfait** s'il sature tous les sommets

Proposition 15:
Si M est parfait, il est maximum.
Si M est maximum, il est maximal.
Toutes les réciproques sont fausses.

Démonstation: Si M est maximum, supposons par l'absurde qu'il n'est pas maximal.
Alors il existe une arête (x,y) telle que ni x ni y n'est saturé par M, et alors M ∪ {(x,y)} est un couplage de cardinal |M| +1.

Réciproque fausse puisque le graphe bleu dans l'exemple suivant est maximal mais pas maximum

Exemple 11:

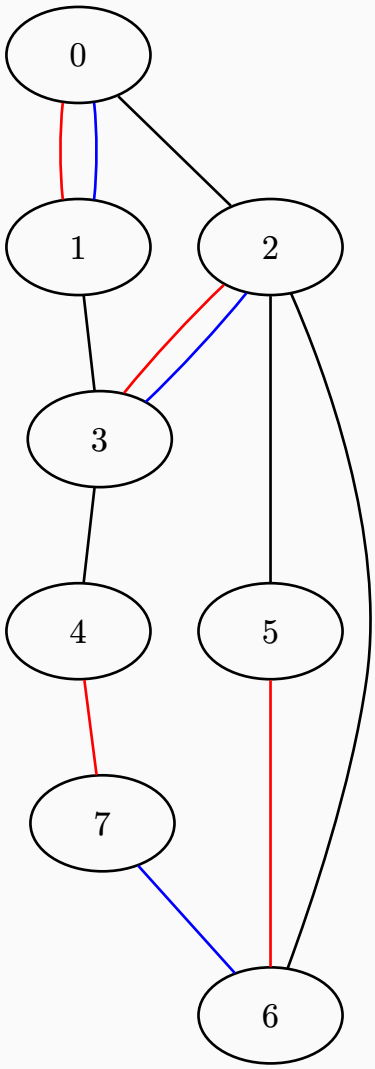


Fig. 18. – Bleu est de taille 3, sature 7 mais pas 4. M est maximal mais non maximum. Rouge est parfait

V.2 Algorithme de Berge (de calcul de couplage maximum)

Tout ce qui est dit ici est valable dans un graphe quelconque.

Définition

Soit M un couplage de G = (V,E) et un chemin simple C dans G.

- On dit que le chemin C est **M-alternant**, **M-alterné** ou **alternant** (par rapport à M) si ses arêtes sont alternativement dans M et pas dans M.
- On dit que C est un chemin **M-augmentant** si ce chemin est M-alternant et que ses extrémités ne sont pas saturées.

Exemple 12:

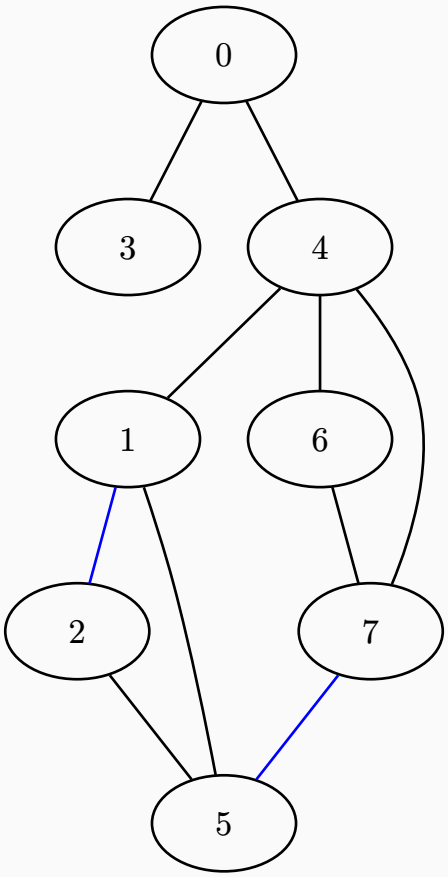


Fig. 19. – Le chemin (1,2,5,7,6) est M-alternant, pas M-augmentant.
Le chemin (4,1,2,5,7,6) est M-augmentant

Proposition 16: Si M est un couplage et C est M-augmentant, alors MΔC est i, couplage de taille |M| + 1.

Intuitivement, celà revient à inverser les arêtes étant dans M et pas dans M le long du chemin.

Démonstation:

Fig. 20. – Les sommets des extrémités n'étant pas saturés, cette modification ne casse pas le couplage.

Lemme: **de Berge**
Soit M un couplage dans un graphe G.

M est maximum si et seulement si il n'xiste pas de chemin M-augmentant dans G

Démonstation: On montre que M n'est pas maxiumum si et seulement s'il existe un chemin M-augmentant

($\Longleftarrow$) Si C est M-augmentant
|MΔC| = [M]+1 et MΔC est un couplage donc M n'est pas maxiumum

($\Longrightarrow$) Supposons ue M n'est pas maximum
Alors il existe M' un couplage de G tel que |M'| > |M|.
Considérons le graphe G' = (V, MΔM').
On va montrer qu'il existe un chemin M-augmentant dans G' et ce chemin existera dasn G ce qui concluera

cf. Fig. 21.

Dans G', les degré des sommets sont au plus deux car si on avait deg(s) $\geq$ 3, s aurait deux arêtes incidentes provenant d'un même couplage, ce qui ne serait don pas un couplage. Ainsi, les composantes connexes dans G' sont de l'une des trois formes suivantes :
- Sommet isolé
- Chemin simple M-alternant
- Cycle M-alternant

Comme |M'| > |M| et que les cycles de G' contiennent autant d'arête de M que de M', il existe un chemin dans G' contenant une arête de plus dans M' que dans M, et comme le chemin est M-alternant, il est nécessairement de la forme :
$x_0 - x_1 - x_2 - ... - x_k - x_{k+1}$ où $x_0 - x_1$ et $x_k - x_{k+1}$ sont dans M'. Il est donc M-augmentant.

CF cours de Tristan
Fig. 21. – Une illustration pour le preuve

Algorithme de Berge (glouton)

```
couplage_maximum(G = (V,E))=
  M ← vide
  tant qu'il existe un chemin C M-augmentant
    M ← MΔC
  Renvoyer M
```

Proposition 17: Cet algoritmhe est totalement correct

Démonstation:
Il termine car [M] augmente à chaque itération et est majoré par |E| (et même par $\frac{v}{2}$).

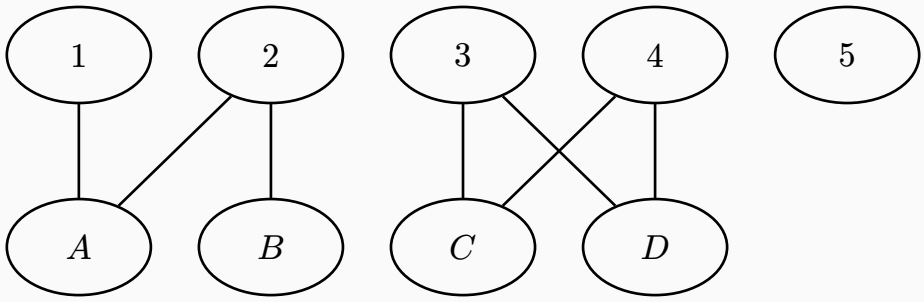
Il est correct car « M est un couplage de G » est invariant pour la boucle tant que. De plus, en fin d'algo, il n'y a plus de chemin M-augmentant donc par le lemme de Berge le couplage est maximum.

V.3 Le cas des graphes bipartis

Définition

Un graphe G = (V,E) est **biparti** s'il existe une bipartition $V = V_1 \sqcup V_2$ telle que toute arête de E connecte un sommet de V_1 et un sommet de V_2

Exemple 13:



Proposition 18: Un graphe est biparti si et seulement si il ne contient aucun cycle de longueur impair

Démonstation: cf. TD

Définition

Si G = ($V_1 \sqcup V_2$, E) est un graphe biparti et M est un couplage de G, on note G_M le graphe orienté tel que :

- Ses sommets sont ceux de G plus deux nouveaux sommets s (source) et d (destination).
- On met un arc depuis s vers chacun des sommets exposés dans V_1.
- On met un arc depuis chacun des sommets exposé de V_2 vers d.
- Les arêtes de G qui sont dans M sont orientés de V_2 vers V_1
- Les arêtes de G qui sont dans E\M sont orientés de V_1 vers V_2.

Proposition 19: $(S, x_0, ..., x_k, d)$ est un chemin dans $G_M$si et seulement si $(x_0, ..., x_k)$ est un chemin M-augmentant dans G

Démonstation:

- Si $s \to x_0 \to y_0 \to ... \to x_k \to y_k \to d$ (où tous les x sont dans V_1 et tous les y sont dans V_2) est un chemin dans G_m et par construction toutes les arêtes d'un y à un x sont dans M et toutes celles d'un x à un y ne sont pas dans M. De plus, x_0 et y_k sont exposés : il est augmentant
- Un chemin M augmentant est nécessairement de longueur impair donc un tel chemin est de la forme $x_0 - y_0 - x_1 - y_1 - ... - x_k - y_k$.
  Comme sa longueur est impaire et que G est biparti, une de ses extrémités est dans V_1, mettons x_0 et l'autre est dans V_2, mettons y_k.

  Comme $x_0 \in V_1$ est exposé, (s, x_0) existe dans G_M
  Comme $y_k \in V_2$ est exposé, (y_k, s) existe dans G_M

  Par bipartition du graphe, les x_i sont dans V_1 et les y_i sont dans V_2.
  Comme il est augmentant, pour tout i, $(x_i, y_i) \notin M$ et $(y_i, x_{i+1}) \in M$

  Ainsi, les (x_i, y_i) et (y_i, x_{i+1}) existent dans G_M.
  Donc (s, x_0, ..., y_k, d) est un chemin dans G_M

Proposition 20: L'algorithme de Berge dans un graphe biparti G = (V,E) peut être implémenté de sorte à avoir une complexité en O(|V|(|V| + |E|))

Démonstation: On fait O(|V|) itérations de la boucle tant que.
Chaque itération coûte

- O(|V| + |E|) pour construire G_M
- O(|V| + |E|) pour trouver un chemin M-augmentant
- O(|E|) pour calculer CΔM